

Free VBA Code + Prompts (Link in Description)

- Prompt:** "Create a VBA macro for Excel that works on a worksheet named "Data".
The macro should be assigned to a button and, when clicked, perform the following tasks without modifying the actual data values: * Automatically detect the full dataset range. * Align all data (including headers) to the center both horizontally and vertically. * Identify and format date values into a consistent format (e.g., dd-mmm-yyyy). * Convert the dataset into a structured Excel Table. * Apply a purple-themed table style, including a colored header, alternating row shading, and visible borders. * Ensure the output is clean, professional, and visually consistent. Only improve formatting and presentation; do not alter or delete any data."

VBA Code: '=====

' DATA CLEANING & FORMATTING MACRO - PURPLE THEME

' Sheet: "Data"

' Columns: Order ID | Order Date | Year | Month | Day |

' Segment | Category | Sub-Category | Region |

' Sales | Profit | Quantity | Discount

'

' BUTTON: Assign this macro -> CleanAndFormatData

' STYLE: Purple theme (TableStyleMedium13)

' Header = Dark purple RGB(94, 59, 183)

' Odd rows = Light purple RGB(235, 228, 255)

' Even rows = White RGB(255, 255, 255)

'=====

Option Explicit

'-----

' MAIN - Assign this to your macro button

'-----

Sub CleanAndFormatData()

```
Application.ScreenUpdating = False
```

```
Application.Calculation = xlCalculationManual
```

```
Dim ws As Worksheet
```

```
Dim lastRow As Long
```

```
Dim lastCol As Long
```

```
Set ws = ThisWorkbook.Sheets("Data")
```

```
' Find actual data boundaries
```

```
lastRow = ws.Cells(ws.Rows.Count, 1).End(xlUp).Row
```

```
lastCol = ws.Cells(1, ws.Columns.Count).End(xlToLeft).Column
```

```
' Run each cleaning & formatting step
```

```
Call StepRemoveBlankRowsCols(ws, lastRow, lastCol)
```

```
Call StepTrimWhitespace(ws, lastRow, lastCol)
```

```
Call StepStandardiseText(ws, lastRow)
```

```
Call StepFormatNumbers(ws, lastRow)
```

```
Call StepCentreAlignment(ws, lastRow, lastCol)
```

```
Call StepApplyPurpleTheme(ws, lastRow, lastCol) ' Purple theme step
```

```
Call StepFormatHeader(ws, lastCol)
```

```
Call StepSetColumnWidths(ws)
```

```
Call StepFreezeHeader(ws)
```

```
Application.Calculation = xlCalculationAutomatic
```

```
Application.ScreenUpdating = True
```

```
MsgBox "Purple Theme Applied Successfully!" & vbNewLine & _
```

```
lastRow - 1 & " records processed.", vbInformation, "Done"
```

```
End Sub
```

```
'-----
```

```
' STEP 1 - Remove entirely blank rows and columns
```

```
'-----
```

```
Sub StepRemoveBlankRowsCols(ws As Worksheet, lastRow As Long, lastCol As Long)
```

```
    Dim r As Long
```

```
    Dim c As Long
```

```
    ' Delete blank rows (bottom-up to avoid row shift issues)
```

```
    For r = lastRow To 2 Step -1
```

```
        If WorksheetFunction.CountA(ws.Rows(r)) = 0 Then
```

```
            ws.Rows(r).Delete
```

```
        End If
```

```
    Next r
```

```
    ' Delete blank columns (right-to-left)
```

```
    For c = lastCol To 1 Step -1
```

```
        If WorksheetFunction.CountA(ws.Columns(c)) = 0 Then
```

```
            ws.Columns(c).Delete
```

```
        End If
```

```
    Next c
```

```
End Sub
```

```
'-----
```

' STEP 2 - Trim leading/trailing spaces from all text cells

'-----

Sub StepTrimWhitespace(ws As Worksheet, lastRow As Long, lastCol As Long)

Dim r As Long

Dim c As Long

Dim cell As Range

For r = 1 To lastRow

For c = 1 To lastCol

Set cell = ws.Cells(r, c)

If cell.Value <> "" And VarType(cell.Value) = vbString Then

cell.Value = Trim(cell.Value)

End If

Next c

Next r

End Sub

'-----

' STEP 3 - Standardise text columns (Proper Case)

' Applies to: Segment, Category, Sub-Category, Region

'-----

Sub StepStandardiseText(ws As Worksheet, lastRow As Long)

Dim r As Long

Dim col As Variant

Dim colNum As Integer

Dim headers As Variant

```
headers = Array("Segment", "Category", "Sub-Category", "Region")
```

```
For Each col In headers
```

```
    colNum = FindColumn(ws, CStr(col))
```

```
    If colNum > 0 Then
```

```
        For r = 2 To lastRow
```

```
            If ws.Cells(r, colNum).Value <> "" Then
```

```
                ws.Cells(r, colNum).Value = _
```

```
                    WorksheetFunction.Proper(ws.Cells(r, colNum).Value)
```

```
            End If
```

```
        Next r
```

```
    End If
```

```
Next col
```

```
End Sub
```

```
'-----
```

```
' STEP 4 - Format number columns with correct formats
```

```
' Sales, Profit -> #,##0.00
```

```
' Quantity    -> #,##0
```

```
' Discount    -> 0%
```

```
' Year, Day    -> 0 (plain integer)
```

```
' Order Date   -> yyyy-mm-dd (convert serials first)
```

```
'-----
```

```
Sub StepFormatNumbers(ws As Worksheet, lastRow As Long)
```

```
    Dim colSales As Integer: colSales = FindColumn(ws, "Sales")
```

```
    Dim colProfit As Integer: colProfit = FindColumn(ws, "Profit")
```

```
    Dim colQty As Integer: colQty = FindColumn(ws, "Quantity")
```

```
Dim colDisc As Integer: colDisc = FindColumn(ws, "Discount")
```

```
Dim colYear As Integer: colYear = FindColumn(ws, "Year")
```

```
Dim colDay As Integer: colDay = FindColumn(ws, "Day")
```

```
Dim colDate As Integer: colDate = FindColumn(ws, "Order Date")
```

```
If colSales > 0 Then ws.Range(ws.Cells(2, colSales), ws.Cells(lastRow,  
colSales)).NumberFormat = "#,##0.00"
```

```
If colProfit > 0 Then ws.Range(ws.Cells(2, colProfit), ws.Cells(lastRow,  
colProfit)).NumberFormat = "#,##0.00"
```

```
If colQty > 0 Then ws.Range(ws.Cells(2, colQty), ws.Cells(lastRow, colQty)).NumberFormat  
= "#,##0"
```

```
If colDisc > 0 Then ws.Range(ws.Cells(2, colDisc), ws.Cells(lastRow,  
colDisc)).NumberFormat = "0%"
```

```
If colYear > 0 Then ws.Range(ws.Cells(2, colYear), ws.Cells(lastRow,  
colYear)).NumberFormat = "0"
```

```
If colDay > 0 Then ws.Range(ws.Cells(2, colDay), ws.Cells(lastRow, colDay)).NumberFormat  
= "0"
```

```
' Order Date: convert Excel serial numbers to readable dates
```

```
If colDate > 0 Then
```

```
    Dim dateCell As Range
```

```
    Dim r As Long
```

```
    For r = 2 To lastRow
```

```
        Set dateCell = ws.Cells(r, colDate)
```

```
        If IsNumeric(dateCell.Value) And dateCell.Value > 1 Then
```

```
            dateCell.Value = CDate(dateCell.Value)
```

```
        End If
```

```
    Next r
```

```
    ws.Range(ws.Cells(2, colDate), ws.Cells(lastRow, colDate)).NumberFormat =  
"yyyy-mm-dd"
```

End If

End Sub

```
'-----  
' STEP 5 - Centre-align ALL cells (header + data)  
'-----
```

Sub StepCentreAlignment(ws As Worksheet, lastRow As Long, lastCol As Long)

With ws.Range(ws.Cells(1, 1), ws.Cells(lastRow, lastCol))

.HorizontalAlignment = xlCenter

.VerticalAlignment = xlCenter

End With

' Uniform row height

ws.Rows("1:" & lastRow).RowHeight = 18

End Sub

```
'-----  
' STEP 6 - Apply Purple Theme (TableStyleMedium13)  
'      Matches your ApplyPurpleTheme macro exactly,  
'      integrated into the full cleaning workflow  
'-----
```

Sub StepApplyPurpleTheme(ws As Worksheet, lastRow As Long, lastCol As Long)

Dim tbl As ListObject

' Detect data range automatically (same as your original macro)

Dim rng As Range

```

Set rng = ws.Range("A1").CurrentRegion

' Unlist any existing tables (removes table format, KEEPS data intact)
On Error Resume Next
For Each tbl In ws.ListObjects
    tbl.Unlist ' FIX: Unlist preserves data; Delete would erase it
Next tbl
On Error GoTo 0

' Create table (same as your original macro)
Set tbl = ws.ListObjects.Add(xlSrcRange, rng, , xlYes)
tbl.Name = "tblSalesData"

' Apply Purple Table Style (your chosen style)
tbl.TableStyle = "TableStyleMedium13"

' Show filter dropdowns
tbl.ShowAutoFilter = True

' ---- Override row banding with exact purple shades ----
' Odd rows -> light purple tint
' Even rows -> white
Dim r As Long
Dim oddBg As Long: oddBg = RGB(235, 228, 255) ' Light purple
Dim evenBg As Long: evenBg = RGB(255, 255, 255) ' White

For r = 2 To lastRow
    With ws.Range(ws.Cells(r, 1), ws.Cells(r, lastCol)).Interior

```



```

    If (r Mod 2) = 1 Then ' Odd row -> light purple
        .Color = oddBg
    Else ' Even row -> white
        .Color = evenBg
    End If
End With

Next r

' Autofit columns (same as your original macro)
rng.Columns.AutoFit

End Sub

'-----
' STEP 7 - Format header row: purple theme styling
'     Dark purple background, white bold text, centred
'-----

Sub StepFormatHeader(ws As Worksheet, lastCol As Long)
    With ws.Range(ws.Cells(1, 1), ws.Cells(1, lastCol))
        .Font.Bold = True
        .Font.Size = 11
        .Font.Color = RGB(255, 255, 255) ' White text
        .Font.Name = "Calibri"
        .Interior.Color = RGB(94, 59, 183) ' Dark purple header
        .HorizontalAlignment = xlCenter
        .VerticalAlignment = xlCenter
        .RowHeight = 22
    End With

```

End Sub

```
'-----  
' STEP 8 - Set column widths tailored to each column  
'-----
```

Sub StepSetColumnWidths(ws As Worksheet)

Dim colMap As Variant

Dim i As Integer

Dim colNum As Integer

```
colMap = Array( _  
    "Order ID", 13, _  
    "Order Date", 14, _  
    "Year", 8, _  
    "Month", 9, _  
    "Day", 7, _  
    "Segment", 14, _  
    "Category", 16, _  
    "Sub-Category", 16, _  
    "Region", 10, _  
    "Sales", 12, _  
    "Profit", 12, _  
    "Quantity", 10, _  
    "Discount", 10)
```

For i = 0 To UBound(colMap) - 1 Step 2

colNum = FindColumn(ws, CStr(colMap(i)))

```

        If colNum > 0 Then
            ws.Columns(colNum).ColumnWidth = CDbl(colMap(i + 1))
        End If
    Next i
End Sub

```

```

'-----
' STEP 9 - Freeze the header row
'-----

```

```

Sub StepFreezeHeader(ws As Worksheet)

    ws.Activate

    ActiveWindow.FreezePanes = False

    ws.Range("A2").Select

    ActiveWindow.FreezePanes = True

    ws.Range("A1").Select

End Sub

```

```

'-----
' HELPER - Find column number by header name (case-insensitive)
'-----

```

```

Function FindColumn(ws As Worksheet, headerName As String) As Integer

    Dim c    As Integer

    Dim lastCol As Integer

    lastCol = ws.Cells(1, ws.Columns.Count).End(xlToLeft).Column

    For c = 1 To lastCol

```

```

    If LCase(Trim(ws.Cells(1, c).Value)) = LCase(Trim(headerName)) Then
        FindColumn = c
        Exit Function
    End If
Next c

FindColumn = 0 ' Not found
End Function

'=====
' STANDALONE: Run this alone to apply only the purple theme
' (Your original macro - preserved as-is)
'=====

Sub ApplyPurpleTheme()
    Dim ws As Worksheet
    Dim tbl As ListObject
    Dim rng As Range
    Set ws = ActiveSheet

    ' Detect data range automatically
    Set rng = ws.Range("A1").CurrentRegion

    ' Unlist existing table if already exists (keeps data, removes table format)
    On Error Resume Next
    For Each tbl In ws.ListObjects
        tbl.Unlist ' FIX: Unlist preserves data; Delete would erase it
    Next tbl

```

On Error GoTo 0

' Create table

Set tbl = ws.ListObjects.Add(xlSrcRange, rng, , xlYes)

' Apply Purple Table Style

tbl.TableStyle = "TableStyleMedium13" ' Purple theme

' Optional: Center align

rng.HorizontalAlignment = xlCenter

' Autofit columns

rng.Columns.AutoFit

MsgBox "Data Clean&Format Successfully!"

End Sub

- 2. Prompt:** "I have an Excel sheet named 'Data' containing raw data. I want a button that, when clicked, automatically creates multiple PivotTables in a separate sheet named 'Pivot'. The PivotTables should be arranged side by side with proper spacing, include clear titles, and display key insights such as sales by segment, trend analysis, monthly performance, category-wise sales, region-wise sales, discount impact, and top-performing orders. The layout should be clean, non-overlapping, and formatted for a professional dashboard view."

VBA Code: '-----

' STEP 2 - All 8 PivotTables side by side in "Pivot" sheet

,

' Layout (all on ROW 1, each in its own column block):

,

' PT_Segment -> A1 (2 cols wide)

```

' PT_Trend    -> D1  (3 cols wide)
' PT_Monthly  -> H1  (5+ cols wide, years as columns)
' PT_Category -> N1  (2 cols wide)
' PT_Region   -> Q1  (2 cols wide)
' PT_Discount -> T1  (3 cols wide)
' PT_Top5     -> X1  (2 cols wide)
'

```

```

' Each block has a 1-column gap to prevent overlap.
'-----

```

```

Sub CreatePivotSheets()

```

```

    Dim wsData As Worksheet

```

```

    Dim wsPvt As Worksheet

```

```

    Dim pc As PivotCache

```

```

    Dim pt As PivotTable

```

```

    Dim lastRow As Long

```

```

    Dim sField As String

```

```

' --- Validate Data sheet ---

```

```

Set wsData = Nothing

```

```

On Error Resume Next

```

```

Set wsData = ThisWorkbook.Sheets("Data")

```

```

On Error GoTo 0

```

```

If wsData Is Nothing Then

```

```

    MsgBox "Sheet named 'Data' not found!", vbCritical

```

```

    Exit Sub

```

```

End If

```

```

' --- Get existing "Pivot" sheet ---

```

```

Set wsPvt = Nothing

On Error Resume Next

Set wsPvt = ThisWorkbook.Sheets("Pivot")

On Error GoTo 0

If wsPvt Is Nothing Then

    MsgBox "Sheet named 'Pivot' not found!" & vbNewLine & _
        "Please create a sheet named 'Pivot' first.", vbCritical

    Exit Sub

End If


' --- Clear all existing PivotTables and content (clean re-run) ---

Dim existPT As PivotTable

For Each existPT In wsPvt.PivotTables

    existPT.TableRange2.Clear

Next existPT

wsPvt.Cells.Clear


' --- Build shared PivotCache ---

lastRow = wsData.Cells(wsData.Rows.Count, 1).End(xlUp).Row

Set pc = ThisWorkbook.PivotCaches.Create( _
    SourceType:=xlDatabase, _
    SourceData:="Data!$A$1:$M$" & lastRow)


' =====

' PT_Segment â€” A1 (2 cols: Row Labels | Total Sales)

' =====

Set pt = pc.CreatePivotTable( _
    TableDestination:=wsPvt.Range("A2"), _

```

```

        TableName:="PT_Segment")

With pt

    .PivotFields("Segment").Orientation = xlRowField

    .PivotFields("Segment").Position = 1

With .PivotFields("Sales")

    .Orientation = xlDataField

    .Function = xlSum

    .NumberFormat = "#,##0.00"

    .Name = "Total Sales"

End With

End With

' =====
' PT_Trend â€” D1 (3 cols: Year | Month | Total Sales | Total Profit)
' 1-col gap (col C) separates from PT_Segment
' =====

Set pt = pc.CreatePivotTable( _
    TableDestination:=wsPvt.Range("D2"), _
    TableName:="PT_Trend")

With pt

    .PivotFields("Year").Orientation = xlRowField

    .PivotFields("Year").Position = 1

    .PivotFields("Month").Orientation = xlRowField

    .PivotFields("Month").Position = 2

With .PivotFields("Sales")

    .Orientation = xlDataField

    .Function = xlSum

    .NumberFormat = "#,##0.00"

```



```

        .Name = "Total Sales"

End With

With .PivotFields("Profit")

    .Orientation = xlDataField

    .Function = xlSum

    .NumberFormat = "#,##0.00"

    .Name = "Total Profit"

End With

End With

' =====
' PT_Monthly â€” H1 (cols: Month | 2023 | 2024 | 2025 | 2026...)
' 1-col gap (col G) separates from PT_Trend
' =====

Set pt = pc.CreatePivotTable( _
    TableDestination:=wsPvt.Range("H2"), _
    TableName:="PT_Monthly")

With pt

    .PivotFields("Month").Orientation = xlRowField

    .PivotFields("Month").Position = 1

    .PivotFields("Year").Orientation = xlColumnField

    .PivotFields("Year").Position = 1

    With .PivotFields("Sales")

        .Orientation = xlDataField

        .Function = xlSum

        .NumberFormat = "#,##0.00"

        .Name = "Total Sales"

    End With

End With

```

End With

```
' =====  
' PT_Category &" N1 (2 cols: Row Labels | Total Sales)  
' 1-col gap (col M) separates from PT_Monthly  
' =====
```

```
Set pt = pc.CreatePivotTable( _  
    TableDestination:=wsPvt.Range("O2"), _  
    TableName:="PT_Category")
```

With pt

```
.PivotFields("Category").Orientation = xlRowField  
.PivotFields("Category").Position = 1  
With .PivotFields("Sales")  
    .Orientation = xlDataField  
    .Function = xlSum  
    .NumberFormat = "#,##0.00"  
    .Name = "Total Sales"
```

End With

End With

```
' =====  
' PT_Region &" Q1 (2 cols: Row Labels | Total Sales)  
' 1-col gap (col P) separates from PT_Category  
' =====
```

```
Set pt = pc.CreatePivotTable( _  
    TableDestination:=wsPvt.Range("R2"), _  
    TableName:="PT_Region")
```

With pt

```

.PivotFields("Region").Orientation = xlRowField
.PivotFields("Region").Position = 1
With .PivotFields("Sales")
    .Orientation = xlDataField
    .Function = xlSum
    .NumberFormat = "#,##0.00"
    .Name = "Total Sales"
End With
End With

' =====
' PT_Discount â€” T1 (3 cols: Sub-Category | Total Profit | Avg Discount)
' 1-col gap (col S) separates from PT_Region
' =====

Set pt = pc.CreatePivotTable( _
    TableDestination:=wsPvt.Range("U2"), _
    TableName:="PT_Discount")
With pt
    .PivotFields("Sub-Category").Orientation = xlRowField
    .PivotFields("Sub-Category").Position = 1
    With .PivotFields("Profit")
        .Orientation = xlDataField
        .Function = xlSum
        .NumberFormat = "#,##0.00"
        .Name = "Total Profit"
    End With
    With .PivotFields("Discount")
        .Orientation = xlDataField

```

```

        .Function = xlAverage
        .NumberFormat = "0.00%"
        .Name = "Avg Discount"
    End With
End With

' =====
' PT_Discount â€” T1 (3 cols: Sub-Category | Total Profit | Avg Discount)
' 1-col gap (col S) separates from PT_Region
' =====

```

```

Set pt = pc.CreatePivotTable( _
    TableDestination:=wsPvt.Range("AB2"), _
    TableName:="PT_Quantity")
With pt
    .PivotFields("Segment").Orientation = xlRowField
    .PivotFields("Segment").Position = 1
    With .PivotFields("Quantity")
        .Orientation = xlDataField
        .Function = xlSum
        .NumberFormat = "#,##0.00"
        .Name = "Total Quantity"
    End With
End With

```

```

' =====
' PT_Top5 " X1 (2 cols: Order ID | Total Sales)
' 1-col gap (col W) separates from PT_Discount
' FIX: AutoSort OUTSIDE With block, runtime field name
' =====

Set pt = pc.CreatePivotTable( _
    TableDestination:=wsPvt.Range("Y2"), _
    TableName:="PT_Top5")

With pt
    .PivotFields("Order ID").Orientation = xlRowField
    .PivotFields("Order ID").Position = 1
    With .PivotFields("Sales")
        .Orientation = xlDataField
        .Function = xlSum
        .NumberFormat = "#,##0.00"
        .Name = "Total Sales"
    End With
End With

sField = pt.DataFields(1).Name
pt.PivotFields("Order ID").AutoSort xlDescending, sField

' --- Write titles above each PivotTable in row 1 ---
Call WritePivotTitles(wsPvt)

' --- Auto-fit all used columns ---
wsPvt.Columns("A:Z").AutoFit

```

' --- Freeze top row so headers stay visible when scrolling ---

wsPvt.Activate

wsPvt.Range("A3").Select

ActiveWindow.FreezePanels = True

End Sub

'-----

' HELPER - Write bold purple title above each PivotTable

'-----

Sub WritePivotTitles(wsPvt As Worksheet)

Dim purple As Long: purple = RGB(94, 59, 183)

Dim bgColor As Long: bgColor = RGB(235, 228, 255)

Dim white As Long: white = RGB(255, 255, 255)

' Define: title text, start col letter, end col letter

Dim titles(7, 2) As String

titles(0, 0) = "Sales by Segment": titles(0, 1) = "A": titles(0, 2) = "B"

titles(1, 0) = "Sales & Profit Trend": titles(1, 1) = "D": titles(1, 2) = "F"

titles(2, 0) = "Monthly Sales Performance": titles(2, 1) = "H": titles(2, 2) = "M"

titles(3, 0) = "Sales by Category": titles(3, 1) = "O": titles(3, 2) = "P"

titles(4, 0) = "Sales by Region": titles(4, 1) = "R": titles(4, 2) = "S"

titles(5, 0) = "Profit vs Discount": titles(5, 1) = "U": titles(5, 2) = "W"

titles(6, 0) = "Top 5 Sales Orders": titles(6, 1) = "Y": titles(6, 2) = "Z"

titles(7, 0) = "Sales by Quantity": titles(7, 1) = "AB": titles(7, 2) = "AC"

Dim i As Integer

For i = 0 To 7

Dim titleRange As String

titleRange = titles(i, 1) & "1:" & titles(i, 2) & "1"

' Merge title cells across pivot width

With wsPvt.Range(titleRange)

.Merge

.Value = titles(i, 0)

.Font.Bold = True

.Font.Size = 14

.Font.Color = white

.Font.Name = "Calibri"

.Interior.Color = purple

.HorizontalAlignment = xlCenter

.VerticalAlignment = xlCenter

.RowHeight = 20

End With

' Bottom border under title to visually separate from pivot

wsPvt.Range(titleRange).Borders(xlEdgeBottom).LineStyle = xlContinuous

wsPvt.Range(titleRange).Borders(xlEdgeBottom).Color = bgColor

wsPvt.Range(titleRange).Borders(xlEdgeBottom).Weight = xlThin

Next i

' Set row 1 height

wsPvt.Rows(1).RowHeight = 20

End Sub

```
'=====
' UTILITY: Refresh all PivotTables (run after data changes)
'=====
```

3. Prompt: "I want to create a fully automated Excel dashboard using a dataset from a 'Data' sheet and summary insights from a 'Pivot' sheet. The dashboard should be generated in a separate 'Dashboard' sheet and should completely rebuild itself whenever I click a 'Refresh Dashboard' button.

The process should:

- Refresh all data sources and calculations
- Update all summary tables
- Recreate the dashboard layout from scratch

The dashboard design should include:

- A top header banner with a title
- KPI cards with key business metrics
- Multiple charts placed in a grid layout (top and bottom sections)
- Proper spacing, alignment, and no overlapping visuals
- Consistent colour theme (purple-based) with professional formatting

The final result should look like a modern business dashboard and be easy to read, interactive, and fully refreshable."

VBA Code: '-----

```
' MAIN ENTRY â€” for first-time build
```

```
'-----
```

```
Sub BuildDashboard()
```

```
    Call DashboardMain
```

```
    MsgBox "Dashboard created successfully!", vbInformation, "Done"
```

```
End Sub
```


'-----

' CORE LOGIC

' FIX 2: Never deletes the Dashboard sheet.

' Finds or creates it, then clears content/charts

' and rebuilds everything in-place.

'-----

Private Sub DashboardMain()

 'Application.ScreenUpdating = False

 'Application.DisplayAlerts = False

 'Application.Calculation = xlCalculationManual

 ' Validate Pivot sheet

 Dim wsPvt As Worksheet

 Set wsPvt = Nothing

 On Error Resume Next

 Set wsPvt = ThisWorkbook.Sheets("Pivot")

 On Error GoTo 0

 If wsPvt Is Nothing Then

 MsgBox "Sheet 'Pivot' not found!", vbCritical

 GoTo CleanUp

 End If

 ' FIX 2: Get existing Dashboard sheet OR create it once

 Dim ws As Worksheet

 Set ws = Nothing

 On Error Resume Next

 Set ws = ThisWorkbook.Sheets("Dashboard")

 On Error GoTo 0

If ws Is Nothing Then

 ' First run "€" create the sheet

 'Set ws = ThisWorkbook.Sheets.Add(Before:=ThisWorkbook.Sheets(1))

 ws.Name = "Dashboard"

 ws.Tab.Color = RGB(94, 59, 183)

End If

 ' FIX 2: Clear existing content and charts in-place (preserves sheet)

 ws.Cells.Clear

 ws.Cells.Interior.ColorIndex = xlNone

 Dim chtObj As ChartObject

 For Each chtObj In ws.ChartObjects

 chtObj.Delete

 Next chtObj

 ' Remove existing button shapes (will be re-added)

 'Dim shp As Shape

 'For Each shp In ws.Shapes

 ' shp.Delete

 'Next shp

 ' Rebuild all sections

 Call SetupDashboardLayout(ws)

 Call BuildHeader(ws)

 Call BuildKPICards(ws, wsPvt)

 Call CreateAllCharts(ws, wsPvt)

```
'Call AddMacroButton(ws)
```

```
ws.Activate
```

```
ActiveWindow.DisplayGridlines = False
```

```
ws.Range("A1").Select
```

```
CleanUp:
```

```
Application.Calculation = xlCalculationAutomatic
```

```
Application.DisplayAlerts = True
```

```
Application.ScreenUpdating = True
```

```
End Sub
```

```
'-----
```

```
' Layout: column widths + row heights
```

```
'-----
```

```
Sub SetupDashboardLayout(ws As Worksheet)
```

```
Dim c As Integer
```

```
For c = 1 To 17
```

```
ws.Columns(c).ColumnWidth = 9.5
```

```
Next c
```

```
ws.Rows("1:2").RowHeight = 20 ' Header
```

```
ws.Rows("3:3").RowHeight = 5 ' Spacer
```

```
ws.Rows("4:6").RowHeight = 16 ' KPI rows
```

```
ws.Rows("7:7").RowHeight = 5 ' Spacer
```

```
Dim r As Integer
```

```
For r = 8 To 31
```

```
    ws.Rows(r).RowHeight = 15
```

```
Next r
```

```
ws.Range("A1:Q31").Interior.Color = RGB(248, 247, 255)
```

```
End Sub
```

```
'-----
```

```
' Header banner
```

```
'-----
```

```
Sub BuildHeader(ws As Worksheet)
```

```
    ws.Range("A1:Q2").Merge
```

```
    With ws.Range("A1")
```

```
        .Value = "SALES PERFORMANCE DASHBOARD"
```

```
        .Font.Name = "Calibri"
```

```
        .Font.Size = 22
```

```
        .Font.Bold = True
```

```
        .Font.Color = RGB(255, 255, 255)
```

```
        .Interior.Color = RGB(94, 59, 183)
```

```
        .HorizontalAlignment = xlCenter
```

```
        .VerticalAlignment = xlCenter
```

```
    End With
```

```
End Sub
```

```
'-----
```

```
' KPI Cards
```

```
' FIX 1: All borders (top, sides, bottom) now use the SAME
' style: xlContinuous + xlThin + purpleDeep
' The shadow row keeps its lighter fill but matches
' the border weight of the header row "no heavy
' xlMedium bottom that looked mismatched.
'-----
```

```
Sub BuildKPICards(ws As Worksheet, wsPvt As Worksheet)
```

```
Dim purple As Long: purple = RGB(94, 59, 183)
```

```
Dim purpleDeep As Long: purpleDeep = RGB(60, 30, 130)
```

```
Dim bgWhite As Long: bgWhite = RGB(255, 255, 255)
```

```
Dim shadow As Long: shadow = RGB(210, 205, 230) ' lighter shadow
```

```
Dim textDark As Long: textDark = RGB(30, 30, 30)
```

```
Dim kpiLabels(5) As String
```

```
Dim kpiFmlas(5) As String
```

```
Dim kpiFmts(5) As String
```

```
Dim kpiCols(5) As Integer
```

```
kpiLabels(0) = "Total Sales"
```

```
kpiLabels(1) = "Total Profit"
```

```
kpiLabels(2) = "Total Orders"
```

```
kpiLabels(3) = "Total Qty Sold"
```

```
kpiLabels(4) = "Avg Discount"
```

```
kpiLabels(5) = "Profit Margin"
```

```
'kpiFmlas(0) = "=Pivot!B6"
```

```
'kpiFmlas(1) = "=Pivot!F46"
```

```
'kpiFmlas(2) = "=COUNTA(Data!A2:A10000)"
```

kpiFmlas(3) = "=SUM(Data!L2:L10000)"

kpiFmlas(4) = "=Pivot!W20"

kpiFmlas(5) = "=Pivot!F46/Pivot!B6"

kpiFmlas(0) = "=GETPIVOTDATA(\"\"Total Sales\"\",Pivot!\$D\$2)"

kpiFmlas(1) = "=GETPIVOTDATA(\"\"Total Profit\"\",Pivot!\$D\$2)"

kpiFmlas(2) = "=COUNTA(Pivot!Y:Y)"

kpiFmlas(3) = "=GETPIVOTDATA(\"\"Quantity\"\",Pivot!\$AB\$2)"

kpiFmlas(4) = "=GETPIVOTDATA(\"\"Avg Discount\"\",Pivot!\$U\$2)"

kpiFmlas(5) = "=D5/B5"

kpiFmts(0) = "[\$\$-en-US]#,##0"

kpiFmts(1) = "[\$\$-en-US]#,##0"

' kpiFmts(0) = ""\$""#,##0"

' kpiFmts(1) = ""\$""#,##0"

kpiFmts(2) = "#,##0"

kpiFmts(3) = "#,##0"

kpiFmts(4) = "0.0%"

kpiFmts(5) = "0.0%"

kpiCols(0) = 2: kpiCols(1) = 4: kpiCols(2) = 6

kpiCols(3) = 8: kpiCols(4) = 10: kpiCols(5) = 12

Dim k As Integer

For k = 0 To 5

Dim c As Integer: c = kpiCols(k)


```

        .Borders(xlEdgeBottom).LineStyle = xlContinuous
        .Borders(xlEdgeBottom).Weight = xlThin      ' FIX 1: was xlMedium
        .Borders(xlEdgeBottom).Color = purpleDeep  ' FIX 1: same as top
        .Borders(xlEdgeLeft).LineStyle = xlContinuous
        .Borders(xlEdgeLeft).Weight = xlThin
        .Borders(xlEdgeLeft).Color = purpleDeep
        .Borders(xlEdgeRight).LineStyle = xlContinuous
        .Borders(xlEdgeRight).Weight = xlThin      ' FIX 1: was xlMedium
        .Borders(xlEdgeRight).Color = purpleDeep

    End With

    Next k

End Sub

```

```

'-----
' CREATE ALL CHARTS
'-----

Sub CreateAllCharts(ws As Worksheet, wsPvt As Worksheet)

    Dim cPurple As Long: cPurple = RGB(94, 59, 183)
    Dim cPink As Long: cPink = RGB(197, 90, 240)
    Dim cOrange As Long: cOrange = RGB(255, 140, 0)
    Dim cBlue As Long: cBlue = RGB(68, 114, 196)
    Dim cWhite As Long: cWhite = RGB(255, 255, 255)

    Dim L1 As Double: L1 = ws.Cells(8, 1).Left
    Dim L2 As Double: L2 = ws.Cells(8, 6).Left
    Dim L3 As Double: L3 = ws.Cells(8, 12).Left
    Dim T1 As Double: T1 = ws.Cells(8, 1).Top

```

Dim T2 As Double: T2 = ws.Cells(19, 1).Top

Dim CW1 As Double: CW1 = ws.Cells(8, 6).Left - ws.Cells(8, 1).Left - 3

Dim CW2 As Double: CW2 = ws.Cells(8, 12).Left - ws.Cells(8, 6).Left - 3

Dim CW3 As Double: CW3 = ws.Cells(8, 17).Left - ws.Cells(8, 12).Left - 3

Dim CH As Double: CH = ws.Cells(19, 1).Top - ws.Cells(8, 1).Top - 3

Dim cht As ChartObject

Dim sc As Integer

Dim p As Integer

' CHART 1 "Sales by Segment (Doughnut)

Set cht = ws.ChartObjects.Add(Left:=L1, Top:=T1, Width:=CW1, Height:=CH)

With cht.Chart

.SetSourceData Source:=wsPvt.Range("A3:B5")

.ChartType = xlDoughnut

.HasTitle = True

.ChartTitle.Text = "Sales by Segment"

Call ApplyTitleStyle(.ChartTitle, cPurple)

Call ApplyChartAreaStyle(cht.Chart)

On Error Resume Next

.SeriesCollection(1).Points(1).Interior.Color = cOrange

.SeriesCollection(1).Points(2).Interior.Color = cPurple

.SeriesCollection(1).Points(3).Interior.Color = cPink

.SeriesCollection(1).HasDataLabels = True

.SeriesCollection(1).DataLabels.ShowPercentage = True

.SeriesCollection(1).DataLabels.ShowValue = False

.SeriesCollection(1).DataLabels.Font.Size = 9

.SeriesCollection(1).DataLabels.Font.Bold = True

```

.SeriesCollection(1).DataLabels.Font.Color = cWhite
On Error GoTo 0
.HasLegend = True
.Legend.Position = xlLegendPositionRight
.Legend.Font.Size = 8
.ShowAllFieldButtons = False
End With

' CHART 2 – Sales & Profit Trend (Line)
Set cht = ws.ChartObjects.Add(Left:=L2, Top:=T1, Width:=CW2, Height:=CH)
With cht.Chart
    .SetSourceData Source:=wsPvt.Range("D3:F46")
    .ChartType = xlLineMarkers
    .HasTitle = True
    .ChartTitle.Text = "Sales & Profit Trend"
    Call ApplyTitleStyle(.ChartTitle, cPurple)
    Call ApplyChartAreaStyle(cht.Chart)
    On Error Resume Next
    With .SeriesCollection(1)
        .Name = "Total Sales"
        .Format.Line.ForeColor.ObjectThemeColor = msoThemeColorAccent2
        .Format.Line.ForeColor.Brightness = 0
        .Format.Line.Weight = 2
        .MarkerStyle = xlMarkerStyleCircle
        .MarkerSize = 5
        .Format.MarkerFill.ForeColor.ObjectThemeColor = msoThemeColorAccent2
        .Format.MarkerLine.ForeColor.ObjectThemeColor = msoThemeColorAccent2
    End With
End With

```

With .SeriesCollection(2)

.Name = "Total Profit"

.Format.Line.ForeColor.ObjectThemeColor = msoThemeColorAccent6

.Format.Line.ForeColor.Brightness = 0

.Format.Line.Weight = 2

.MarkerStyle = xlMarkerStyleCircle

.MarkerSize = 5

.Format.MarkerFill.ForeColor.ObjectThemeColor = msoThemeColorAccent6

.Format.MarkerLine.ForeColor.ObjectThemeColor = msoThemeColorAccent6

End With

On Error GoTo 0

.HasLegend = True

.Legend.Position = xlLegendPositionBottom

.Legend.Font.Size = 8

.Axes(xlValue).TickLabels.NumberFormat = "#,##0"

.Axes(xlValue).TickLabels.Font.Size = 8

.Axes(xlCategory).TickLabels.Font.Size = 7

Call RemoveGridlines(cht.Chart)

.ShowAllFieldButtons = False

End With

' CHART 3 – Monthly Sales Performance (Column)

Set cht = ws.ChartObjects.Add(Left:=L3, Top:=T1, Width:=CW3, Height:=CH)

With cht.Chart

.SetSourceData Source:=wsPvt.Range("H3:L15")

.ChartType = xlColumnClustered

.HasTitle = True

.ChartTitle.Text = "Monthly Sales Performance"

```

Call ApplyTitleStyle(.ChartTitle, cPurple)
Call ApplyChartAreaStyle(cht.Chart)
On Error Resume Next
Dim yearCols(3) As Long
yearCols(0) = cPurple
yearCols(1) = cPink
yearCols(2) = cOrange
yearCols(3) = cBlue
For sc = 1 To .SeriesCollection.Count
    If sc <= 4 Then .SeriesCollection(sc).Interior.Color = yearCols(sc - 1)
Next sc
On Error GoTo 0
.HasLegend = True
.Legend.Position = xlLegendPositionBottom
.Legend.Font.Size = 8
.Axes(xlValue).TickLabels.NumberFormat = "#,##0"
.Axes(xlValue).TickLabels.Font.Size = 8
.Axes(xlCategory).TickLabels.Font.Size = 8
Call RemoveGridlines(cht.Chart)
.ShowAllFieldButtons = False
End With

' CHART 4 – Profit vs Discount (Column)
Set cht = ws.ChartObjects.Add(Left:=L1, Top:=T2, Width:=CW1, Height:=CH)
With cht.Chart
    .SetSourceData Source:=wsPvt.Range("U3:V19")
    .ChartType = xlColumnClustered
    .HasTitle = True

```

```

.ChartTitle.Text = "Profit vs Discount"
Call ApplyTitleStyle(.ChartTitle, cPurple)
Call ApplyChartAreaStyle(cht.Chart)
On Error Resume Next

.SeriesCollection(1).Interior.Color = cPink

.SeriesCollection(1).Name = "Total Profit"
On Error GoTo 0

.HasLegend = False

.Axes(xlValue).TickLabels.NumberFormat = "#,##0"

.Axes(xlValue).TickLabels.Font.Size = 8

.Axes(xlCategory).TickLabels.Font.Size = 6

.Axes(xlCategory).TickLabelSpacing = 1

Call RemoveGridlines(cht.Chart)

.ShowAllFieldButtons = False

End With

' CHART 5 – Sales by Category (Bar)
Set cht = ws.ChartObjects.Add(Left:=L2, Top:=T2, Width:=CW2, Height:=CH)
With cht.Chart

.SetSourceData Source:=wsPvt.Range("O3:P5")

.ChartType = xlBarClustered

.HasTitle = True

.ChartTitle.Text = "Sales by Category"

Call ApplyTitleStyle(.ChartTitle, cPurple)

Call ApplyChartAreaStyle(cht.Chart)

On Error Resume Next

.SeriesCollection(1).Points(1).Interior.Color = cPink

.SeriesCollection(1).Points(2).Interior.Color = cPurple

```

```

.SeriesCollection(1).Points(3).Interior.Color = cOrange
.SeriesCollection(1).HasDataLabels = True
.SeriesCollection(1).DataLabels.ShowValue = True
.SeriesCollection(1).DataLabels.NumberFormat = "#,##0"
.SeriesCollection(1).DataLabels.Font.Size = 8
On Error GoTo 0
.HasLegend = True
.Legend.Position = xlLegendPositionRight
.Legend.Font.Size = 8
.Axes(xlValue).TickLabels.NumberFormat = "#,##0"
.Axes(xlValue).TickLabels.Font.Size = 8
.Axes(xlCategory).TickLabels.Font.Size = 9
Call RemoveGridlines(cht.Chart)
.ShowAllFieldButtons = False
End With

' CHART 6 – Sales by Region (Column)
Set cht = ws.ChartObjects.Add(Left:=L3, Top:=T2, Width:=CW3, Height:=CH)
With cht.Chart
    .SetSourceData Source:=wsPvt.Range("R3:S6")
    .ChartType = xlColumnClustered
    .HasTitle = True
    .ChartTitle.Text = "Sales by Region"
    Call ApplyTitleStyle(.ChartTitle, cPurple)
    Call ApplyChartAreaStyle(cht.Chart)
    On Error Resume Next
    Dim regCols(3) As Long
    regCols(0) = cPurple

```

```

regCols(1) = cPink
regCols(2) = cOrange
regCols(3) = cBlue
For p = 1 To .SeriesCollection(1).Points.Count
    If p <= 4 Then .SeriesCollection(1).Points(p).Interior.Color = regCols(p - 1)
Next p

.SeriesCollection(1).HasDataLabels = True
.SeriesCollection(1).DataLabels.ShowValue = True
.SeriesCollection(1).DataLabels.NumberFormat = "$#,##0"
.SeriesCollection(1).DataLabels.Font.Size = 8

On Error GoTo 0

.HasLegend = False

With .Axes(xlValue)
    .MinimumScaleIsAuto = True
    '.MinimumScale = 2050000
    .MaximumScaleIsAuto = True
    .TickLabels.NumberFormat = "#,##0"
    .TickLabels.Font.Size = 8
End With

.Axes(xlCategory).TickLabels.Font.Size = 9

Call RemoveGridlines(cht.Chart)

.ShowAllFieldButtons = False

End With

End Sub

```

'-----

' MACRO BUTTON "€" top-right of header, stays in same sheet


```
' .OnAction -> RefreshDashboard (rebuilds in-place)
```

```
'-----
```

```
Sub AddMacroButton(ws As Worksheet)
```

```
    ' Button is already deleted in DashboardMain shape-clear loop
```

```
    Dim btnLeft As Double: btnLeft = ws.Cells(1, 15).Left + 4
```

```
    Dim btnTop As Double: btnTop = ws.Cells(1, 1).Top + 4
```

```
    Dim btnWidth As Double: btnWidth = ws.Cells(1, 17).Left + ws.Columns(17).Width -  
ws.Cells(1, 15).Left - 8
```

```
    Dim btnHeight As Double: btnHeight = ws.Cells(3, 1).Top - ws.Cells(1, 1).Top - 8
```

```
    Dim shp As Shape
```

```
    Set shp = ws.Shapes.AddShape(msoShapeRoundedRectangle, _  
                                btnLeft, btnTop, btnWidth, btnHeight)
```

```
    With shp
```

```
        .Name = "btnRefreshDashboard"
```

```
        .Fill.ForeColor.RGB = RGB(255, 255, 255)
```

```
        .Fill.Solid
```

```
        .Line.ForeColor.RGB = RGB(94, 59, 183)
```

```
        .Line.Weight = 1.5
```

```
        .Line.Visible = msoTrue
```

```
        .TextFrame2.TextRange.Text = "Refresh Dashboard"
```

```
        With .TextFrame2.TextRange.Font
```

```
            .Bold = msoTrue
```

```
            .Size = 9
```

```
            .Fill.ForeColor.RGB = RGB(94, 59, 183)
```

```
        End With
```

```
        .TextFrame.HorizontalAlignment = xlHAlignCenter
```

```
        .TextFrame.VerticalAlignment = xlVAlignCenter
```

```
.OnAction = "RefreshDashboard"
```

```
End With
```

```
End Sub
```

```
'-----
```

```
' HELPER: Title styling
```

```
'-----
```

```
Sub ApplyTitleStyle(ct As ChartTitle, titleColor As Long)
```

```
    ct.Font.Bold = True
```

```
    ct.Font.Color = titleColor
```

```
    ct.Font.Size = 11
```

```
    ct.Font.Name = "Calibri"
```

```
End Sub
```

```
'-----
```

```
' HELPER: Chart area styling
```

```
'-----
```

```
Sub ApplyChartAreaStyle(cht As Chart)
```

```
    cht.ChartArea.Interior.Color = RGB(245, 242, 255)
```

```
    With cht.ChartArea.Border
```

```
        .LineStyle = xlContinuous
```

```
        .Weight = xlHairline
```

```
        .Color = RGB(94, 59, 183)
```

```
    End With
```

```
    cht.PlotArea.Interior.ColorIndex = xlNone
```

```
    On Error Resume Next
```

```
    cht.PlotArea.Border.LineStyle = xlNone
```

```
    On Error GoTo 0
```

End Sub

```
'-----  
' HELPER: Remove all gridlines  
'-----
```

Sub RemoveGridlines(cht As Chart)

On Error Resume Next

cht.Axes(xlValue).HasMajorGridlines = False

cht.Axes(xlValue).HasMinorGridlines = False

cht.Axes(xlCategory).HasMajorGridlines = False

cht.Axes(xlCategory).HasMinorGridlines = False

On Error GoTo 0

End Sub

4. Bonus: One-Click Refresh Dashboard:

Option Explicit

Sub RefreshAllPivots()

Dim ws As Worksheet

Dim pt As PivotTable

For Each ws In ThisWorkbook.Worksheets

For Each pt In ws.PivotTables

pt.RefreshTable

Next pt

Next ws

MsgBox "Data & Dashboard refreshed successfully.", vbInformation, "Refresh Complete"

End Sub

